

**SIMATS ENGINEERING**  
**ASSESSMENT TOOL 1 – Application-Based Assessment Question**

|                                         |                      |                                                       |
|-----------------------------------------|----------------------|-------------------------------------------------------|
| <b>Course Name:</b> Ethical Hacking     | <b>Code:</b> ITA1404 | <b>CO:</b> CO5                                        |
| <b>Duration:</b> 90 min<br><b>Date:</b> | <b>Total Marks:</b>  | <b>Reg. No / Name : S Venkata Abhishek(192311053)</b> |

## **COURSE OUTCOME COVERED**

***CO5:** Analyze vulnerabilities in Windows and Linux systems and evaluate security weaknesses in messaging, web, and mobile applications using appropriate exploitation techniques and hacking tools. (BT5)*

## **WINDOWS VULNERABILITIES**

### **1. Anonymous SMB / Null Sessions**

A Windows file server allowing anonymous SMB access creates a security risk because attackers may enumerate users, shares, and other network information without authentication. This information can support further attacks and lateral movement.

The organization should disable Null Sessions immediately if there is no legitimate business dependency. However, if a legacy application requires anonymous access, the dependency should first be identified and tested. Temporary compensating controls such as firewall restrictions, SMB access control, monitoring, and least-privilege permissions can be applied.

Therefore, disabling Null Sessions is the preferred long-term solution because it removes the root cause rather than only reducing its impact.

Tool relevance: Nmap and Enum4linux can be used in an authorized assessment to verify SMB exposure and anonymous enumeration.

---

### **2. SMBv1 Compatibility**

SMBv1 is an outdated protocol with significant security weaknesses and has been associated with major attacks such as WannaCry. Continuing to use it increases the attack surface and exposes systems to known vulnerabilities.

The organization should migrate to SMBv2 or SMBv3. The migration should be performed in stages by identifying systems that depend on SMBv1, testing compatibility, upgrading or replacing legacy applications, and then disabling SMBv1.

Therefore, maintaining SMBv1 only for convenience is not recommended. A controlled migration provides better security while minimizing business disruption.

Tool relevance: Nmap and smbclient can be used to identify SMB services and verify protocol configurations.

---

### **3. Writable Shared Folders**

Writable shared folders accessible by all employees represent a high security risk. Any employee or compromised account may modify, delete, or replace files. Attackers could also use writable locations to distribute malicious files or facilitate malware propagation.

The first priority should be to identify sensitive and business-critical shares. Permissions should then be changed according to the principle of least privilege. Users should receive only the read or write permissions required for their job.

Therefore, unrestricted write access should be removed immediately, especially for critical shares.

Tool relevance: Enum4linux and smbclient can be used during an authorized assessment to identify accessible SMB shares and their permissions.

---

### **4. Exposed RPC Services**

RPC services should not automatically be disabled because some Windows applications and management functions depend on them. However, unnecessary or externally accessible RPC services increase the attack surface.

The organization should identify which RPC services are required for business operations. Unnecessary services should be disabled, while required services should be restricted using firewalls, network segmentation, and access-control policies.

Therefore, the correct approach is to minimize RPC exposure rather than blindly disabling every RPC service.

Tool relevance: Nmap and rpcclient can help identify and assess exposed RPC services during an authorized security assessment.

---

### **5. Brute-Force vs Passive Enumeration**

Passive or low-impact enumeration is more suitable for a production environment because it minimizes the possibility of service disruption, account lockouts, excessive logging, and detection.

Brute-force enumeration can generate large amounts of traffic and may trigger security controls or affect system performance.

Therefore, a penetration tester should initially use passive and low-impact techniques. More intrusive enumeration should only be performed when explicitly authorized and when the operational risk is acceptable.

Tool relevance: Nmap and Enum4linux can support controlled enumeration.

---

## **6. Outdated Windows Server**

Patching an outdated Windows Server is essential, but patching alone is not sufficient to eliminate the overall security risk.

The organization should combine patching with service hardening, removal of unnecessary services, network segmentation, least-privilege access, endpoint protection, vulnerability scanning, and continuous monitoring.

Therefore, patching should be treated as one part of a defense-in-depth strategy rather than the complete security solution.

Tool relevance: Nmap can identify exposed services, while OpenVAS/Greenbone can help identify known vulnerabilities.

---

## **7. Domain Administrator Account Discovered**

A discovered domain administrator account represents a potentially serious security risk because compromise of the account could provide extensive control over the Windows domain.

The security team should immediately verify whether the account is legitimate, determine its privileges, review authentication logs, check for suspicious activity, and monitor for signs of compromise. Excessive privileges should be removed and strong authentication such as MFA should be enforced where supported.

Therefore, the account should not simply be deleted without investigation; its legitimacy, exposure, and activity should first be evaluated.

Tool relevance: Enum4linux and authorized credential/security assessment tools can help identify account exposure.

---

## **8. Guest Accounts vs Strong Authentication**

Disabling unnecessary guest accounts reduces anonymous or unauthenticated access, but it does not protect the organization against attacks using compromised legitimate credentials.

Strong authentication policies provide broader protection because they reduce the likelihood that stolen or weak credentials can be used to access systems.

Therefore, strong authentication should be prioritized while unnecessary guest accounts should also be disabled.

The best approach combines account restrictions, strong passwords, MFA, least privilege, and continuous monitoring.

---

## **9. Unnecessary Windows Services**

Unnecessary services increase the attack surface because each running service may contain vulnerabilities or provide an additional entry point.

The organization should first prioritize services that have known vulnerabilities, are exposed to untrusted networks, or have no legitimate business purpose. These services should be disabled or restricted.

Services required by business operations should be patched, securely configured, and monitored.

Therefore, service removal should be based on both technical risk and business requirements.

Tool relevance: Nmap can identify exposed ports and services.

---

## **10. Exploiting a Vulnerability in Production**

Exploiting vulnerabilities in production can provide valuable evidence about the actual impact of a security weakness, but it also introduces operational risks such as service disruption, data modification, or system instability.

A penetration tester should exploit only within the agreed scope and rules of engagement. The tester should select the least intrusive method that provides sufficient evidence and avoid destructive actions.

Therefore, exploitation is justified only when authorization, business impact, safety controls, and testing boundaries have been clearly established.

Tool relevance: Metasploit Framework can be used in an authorized and controlled penetration-testing environment.

---

## **LINUX VULNERABILITIES**

### **11. Unrestricted NFS Exports**

Unrestricted NFS exports can allow unauthorized systems to access sensitive files. If permissions are poorly configured, attackers may read, modify, or potentially gain higher privileges.

NFS exports should be restricted to trusted IP addresses or networks. Sensitive directories should not be unnecessarily exported, and root squashing should be enabled where appropriate.

Therefore, unrestricted NFS access should be treated as a high-priority configuration weakness.

Tool relevance: showmount and Nmap can be used to identify exposed NFS services and exports during an authorized assessment.

---

### **12. .rhosts Files**

The .rhosts mechanism presents a significant security risk because it can allow trust-based remote access without strong authentication.

Such files should generally be removed from modern production environments. SSH with strong authentication should be used instead.

Therefore, .rhosts should not normally remain enabled unless there is an exceptional and documented legacy requirement.

Tool relevance: find and grep can be used for authorized file-system auditing.

---

### **13. World-Writable Files**

World-writable files can be modified by unauthorized users and may create opportunities for data manipulation, malware insertion, or privilege escalation.

The organization should identify world-writable files, determine whether the permissions are required, and change them to the minimum permissions necessary.

Particular attention should be given to files executed by privileged processes.

Therefore, unnecessary world-writable permissions should be corrected immediately.

Tool relevance: find and ls can be used for permission auditing.

---

### **14. Outdated SSH Version**

Updating SSH may be important, but it should not automatically be considered the highest priority vulnerability.

The security team should evaluate the specific SSH version, known vulnerabilities, exploitability, internet exposure, server criticality, and availability of compensating controls.

If the SSH service has a critical remotely exploitable vulnerability and is internet-facing, it should receive high priority. Otherwise, other vulnerabilities with greater risk may need to be addressed first.

Therefore, remediation should be risk-based rather than based solely on software age.

Tool relevance: Nmap can help identify SSH versions during authorized assessment.

---

### **15. Anonymous FTP**

Anonymous FTP can expose files to unauthenticated users and may allow unauthorized downloading or uploading of information.

It should normally be disabled unless there is a clearly documented business requirement. If it must remain enabled, access should be restricted and sensitive directories should not be exposed.

Therefore, disabling anonymous FTP is the preferred security configuration.

Tool relevance: Nmap and FTP clients can be used in an authorized environment to verify configuration.

---

## **16. Password vs SSH Key Authentication**

Password authentication is vulnerable to guessing, credential theft, password reuse, and brute-force attacks.

SSH key authentication generally provides stronger protection when keys are properly generated, protected with passphrases, and managed securely. MFA can provide an additional layer of protection.

Therefore, enterprise Linux servers should generally prefer SSH keys with strong key management and MFA where practical.

---

## **17. Disabling Unused Linux Services**

Disabling unused Linux services significantly improves security because it reduces the number of active network entry points.

It also reduces unnecessary resource consumption and simplifies system administration.

However, services should not be disabled without confirming that they are not required by applications or business operations.

Therefore, removing unnecessary services is an effective attack-surface-reduction measure.

Tool relevance: systemctl can manage services, while Nmap can verify externally exposed services.

---

## **18. Linux Buffer Overflow**

A buffer overflow can allow unexpected memory modification and potentially lead to application crashes or unauthorized code execution.

The safest remediation is to patch or replace the vulnerable application and correct the underlying programming error. Secure coding practices, bounds checking, compiler protections, ASLR, stack canaries, and other system protections should also be applied.

Therefore, relying only on runtime protection is insufficient; the underlying vulnerability should be corrected.

Tool relevance: GDB and Valgrind can assist authorized developers and security teams in analyzing memory-related defects.

---

## **19. File Permission Audits**

Regular file permission audits are an effective preventive security measure because excessive permissions can allow unauthorized users to access or modify sensitive information.

Audits should identify world-writable files, incorrectly owned files, sensitive files with excessive access, and dangerous privilege configurations.

Therefore, permission auditing should be included in regular Linux security reviews.

Tool relevance: find and stat can help identify file ownership and permission problems.

---

## **20. Automated Scanners vs Manual Assessment**

Automated vulnerability scanners provide broad and repeatable coverage and can quickly identify many known vulnerabilities and configuration weaknesses.

However, scanners may produce false positives, miss business logic issues, and fail to understand application-specific risks.

Manual assessment provides context and can identify weaknesses that automated tools cannot understand.

Therefore, automated scanners should complement rather than replace manual security assessments.

Tool relevance: Greenbone and Lynis can support automated Linux security assessment.

---

## **WEB APPLICATIONS**

### **21. SQL Injection**

Patching the vulnerable code is essential but should not be considered the only security measure.

The application should use parameterized queries or prepared statements, validate input on the server, apply least-privilege database accounts, and use appropriate monitoring and WAF protections.

Therefore, secure coding is the primary defense, while WAF and monitoring provide additional layers of protection.

Tool relevance: OWASP ZAP and sqlmap can be used only against authorized applications to validate SQL injection exposure.

---

### **22. WAF Against XSS**

A Web Application Firewall can detect and block some XSS payloads, reducing the probability or impact of attacks.

However, a WAF is not a replacement for secure application development. Poor input handling and output encoding can still leave the application vulnerable, and attackers may find ways around WAF rules.

Therefore, WAF protection should complement server-side validation, contextual output encoding, secure frameworks, and CSP.

---

### **23. Hidden Admin Pages**

Hidden administrative pages create a security risk if they are accessible without proper authentication and authorization.

Simply hiding a URL does not provide security because attackers may discover it through application behavior, source code, links, or directory enumeration.

Therefore, every administrative function must enforce strong authentication and server-side authorization.

Tool relevance: Gobuster and Dirsearch can be used in authorized testing to identify exposed paths.

---

### **24. Client-Side Validation**

Client-side validation alone is not sufficient because users can modify or bypass client-side code and send requests directly to the server.

All important validation and authorization decisions must therefore be performed on the server.

Client-side validation is useful for improving user experience but should be considered an additional control rather than a security boundary.

Tool relevance: Burp Suite Community and OWASP ZAP can be used to assess server-side validation in authorized applications.

---

### **25. Manual vs Automated Web Testing**

Automated scanning is efficient for discovering common vulnerabilities and configuration problems across large applications.

Manual testing is better for understanding business logic, authorization weaknesses, complex workflows, and vulnerabilities requiring human reasoning.

Therefore, neither approach completely replaces the other. A combination of automated scanning and manual testing provides the strongest coverage.

---

### **26. Weak Session Management**



Weak session management can allow attackers to hijack accounts, reuse stolen sessions, or bypass authentication controls.

Examples include predictable session identifiers, excessive session lifetime, failure to invalidate sessions after logout, and insecure cookie settings.

The organization should use secure random session tokens, HTTPS, Secure and HttpOnly cookie attributes, appropriate expiration, session rotation, and server-side invalidation.

Therefore, weak session management should be treated as a high-priority issue, particularly for e-commerce applications.

---

## **27. Login Without Rate Limiting**

A login page without rate limiting is vulnerable to repeated authentication attempts and credential attacks.

The organization should implement rate limiting, MFA, CAPTCHA or equivalent bot controls where appropriate, monitoring, and risk-based authentication. Account lockout should be designed carefully to avoid creating denial-of-service opportunities.

Therefore, rate limiting is an important control for protecting authentication endpoints.

Tool relevance: Hydra may be used only for explicitly authorized testing, while OWASP ZAP can assist with web security assessment.

---

## **28. HTTPS**

HTTPS protects data while it is transmitted between the client and server, helping prevent interception and modification during transport.

However, HTTPS does not protect against SQL injection, XSS, broken authentication, insecure authorization, malware, or application logic vulnerabilities.

Therefore, HTTPS is essential but does not guarantee overall web application security.

Tool relevance: SSLyze and testssl.sh can help assess TLS configuration in authorized environments.

---

## **29. Content Security Policy**

Content Security Policy can significantly reduce the impact of some XSS attacks by controlling which scripts and resources browsers are allowed to execute.

However, CSP should not be considered a replacement for secure coding. Poorly designed policies can also reduce its effectiveness.

Therefore, CSP should be implemented as an additional defense alongside input validation, output encoding, secure frameworks, and proper session security.

---

### **30. Prioritizing 150 Vulnerabilities**

A vulnerability scanner's number of findings does not determine which issues should be fixed first.

The organization should prioritize findings according to:

1. Severity/CVSS.
2. Exploitability.
3. Internet exposure.
4. Business criticality.
5. Sensitive data involved.
6. Availability of public exploits.
7. Existing compensating controls.

Critical, externally exploitable vulnerabilities affecting sensitive systems should receive the highest priority.

Therefore, risk-based prioritization is more effective than simply fixing vulnerabilities according to their discovery order.

---

## **MOBILE APPLICATIONS**

### **31. Plaintext Password Storage**

Storing passwords in plaintext is a critical security weakness because anyone gaining access to the application's storage or backend data may obtain user credentials directly.

Passwords should never be stored as plaintext. They should be protected using appropriate password hashing algorithms with unique salts and secure parameters.

Sensitive authentication information should also be protected using secure storage mechanisms.

Therefore, plaintext password storage should receive immediate remediation.

Tool relevance: MobSF and JADX can assist in authorized mobile application security analysis.

---

### **32. Certificate Pinning in Banking Applications**

Certificate pinning can strengthen protection against certain man-in-the-middle attacks by allowing an application to verify expected certificates or public keys.

For a banking application handling highly sensitive transactions, certificate pinning can provide an additional security layer.

However, certificate rotation and operational maintenance must be carefully designed to prevent accidental service disruption.

Therefore, certificate pinning is strongly worth considering for high-risk applications when implemented with a robust certificate-management strategy.

---

### **33. Excessive Android Permissions**

Excessive permissions violate the principle of least privilege and increase the potential impact of a compromised application or malicious component.

An application requesting unnecessary access to contacts, location, storage, microphone, or other sensitive resources creates additional privacy and security risks.

Therefore, permissions should be limited to those genuinely required by application functionality.

---

### **34. Static vs Dynamic Android Analysis**

Static analysis examines the application without executing it. It can identify insecure code, hardcoded secrets, dangerous APIs, permissions, and configuration weaknesses.

Dynamic analysis examines application behavior during execution and can reveal runtime communication, storage behavior, authentication issues, and other weaknesses.

Neither method is sufficient by itself.

Therefore, static and dynamic analysis should be used together to achieve broader security coverage.

Tool relevance: MobSF supports mobile security analysis, while JADX can assist with authorized code inspection.

---

### **35. Code Obfuscation**

Code obfuscation makes reverse engineering more difficult by transforming application code into a less understandable form.

However, it does not eliminate vulnerabilities and cannot protect poorly designed authentication, insecure APIs, plaintext secrets, or weak cryptography.

Therefore, obfuscation should be considered a defense-in-depth mechanism rather than the primary security control.

ProGuard/R8 can support Android code optimization and obfuscation.

---

## **DATABASES AND REPORTING**

### **36. Default Database Administrator Credentials**

Default administrator credentials represent a critical security risk because attackers may know or easily discover the default username and password.

If compromised, administrative database access can result in unauthorized data modification, disclosure, or deletion.

The organization should immediately change default credentials, use strong authentication, apply least privilege, restrict database network access, and monitor administrative activity.

Therefore, default credentials should never remain enabled in a production database.

---

### **37. Encryption at Rest vs Encryption in Transit**

Encryption at rest protects stored data such as database files, backups, and storage devices if they are accessed without authorization.

Encryption in transit protects information while it moves between applications, databases, users, and services.

They address different threats and therefore cannot be treated as alternatives.

Therefore, both are essential for protecting customer information.

A strong security architecture should use encryption at rest together with TLS or equivalent secure communication mechanisms.

---

### **38. Prioritizing 25 Vulnerabilities**

The final penetration-testing report should prioritize vulnerabilities based on actual business risk rather than simply listing them by technical severity.

Critical factors include:

- Exploitability.
- Business impact.
- Sensitive data exposure.
- Internet accessibility.
- Privilege required.
- Availability of exploits.
- Existing security controls.

Critical and high-risk vulnerabilities affecting important systems should be presented first, followed by medium- and low-risk findings.

---

### **39. Proof-of-Concept Exploit Code**

Proof-of-concept information can help technical teams understand and reproduce a vulnerability, but exploit code should not automatically be included in every report.

The decision should consider authorization, sensitivity, risk of misuse, technical requirements, and whether the code is actually necessary for remediation.

Therefore, a controlled and minimal demonstration is preferable to providing unnecessary weaponized exploit material.

---

#### **40. Executive Summary vs Technical Report**

An executive summary alone is not sufficient for the technical team because management and technical personnel have different information requirements.

Management generally needs a high-level explanation of business risk, major findings, impact, and recommended priorities.

The technical team needs detailed information such as affected systems, evidence, severity, root cause, remediation guidance, and validation requirements.

Therefore, the final penetration-testing report should contain both:

1. **Executive Summary** – for management and decision-makers.
2. **Technical Report** – for security, IT, and development teams.

This structure ensures that management understands the business risk while technical teams have enough information to remediate the vulnerabilities effectively.